

Does a Proposed Data Validation Pipeline Catch More Invalid Records Than the Legacy Pipeline?

A Paired Binary Classifier Comparison Using McNemar's Test and Bayesian Analysis

Prepared as part of an Inferential Statistics case-study series | Analyses performed in Python (SciPy, statsmodels, pandas) and R (base stats)

Abstract

Background: Organizations often maintain automated data validation pipelines to catch invalid records before they reach downstream systems. This case study evaluates whether a proposed pipeline (B), which adds cross-field and duplicate-detection checks to a legacy pipeline (A), genuinely improves detection accuracy. **Methods:** 800 synthetic order records with realistic fields and injected defects were generated, and both pipelines were implemented as executable rule-checking functions and run against every record — the confusion table below is a measured output of that process, not an assumed input. Prior to analysis, each defect mechanism was individually audited by checking its actual correct/incorrect rate against design intent; this audit caught and corrected a flaw in the duplicate-record generator (an injected whitespace variation was being caught by an unrelated email-format check before duplicate-detection logic ever ran), and the corrected generator is what produced the results reported here. Because the same records are scored by both pipelines, this is a paired binary comparison; McNemar's test was pre-specified as the primary analysis, since only records on which the pipelines disagree carry information about which is more accurate. Two complementary Bayesian analyses were also conducted: a Beta-Binomial model on the disagreements, and a Dirichlet-Multinomial model on the full 2×2 outcome table. **Results:** Pipeline B achieved substantially higher accuracy (96.62% vs. 85.12%). McNemar's exact test found this difference extremely unlikely to be due to chance ($p = 1.16 \times 10^{-18}$). Both Bayesian models estimated a probability greater than 0.999 that Pipeline B is genuinely more accurate, with an expected improvement of 11.44 percentage points (95% credible interval: 8.92 to 14.07). **Conclusion:** Pipeline B should be adopted, conditional on fixing two specific regressions identified during the analysis.

1. Introduction / Background

Automated data validation is a first line of defense against bad data entering downstream analytics and business systems. A common evolution in such pipelines is adding cross-field checks (e.g., verifying that a shipment date follows an order date, or that a postal code matches a stated state) on top of simpler field-level checks (e.g., confirming a field is non-empty or matches a basic format). This case study evaluates such an upgrade using a paired evaluation design against synthetic but mechanistically-generated ground truth, and demonstrates how a rigorous statistical comparison can surface not just whether a system improved, but exactly where its remaining weaknesses lie.

2. Methods

2.1 Synthetic Data Generation

800 order records were generated with realistic fields (order ID, email, order date, ship date, zip code, state, price). Each record was assigned a ground-truth validity label and, for invalid records, one of three defect

mechanisms: **basic defects** (malformed or missing email, or non-positive price — detectable by simple field-level checks); **cross-field defects** (ship date preceding order date, zip/state mismatch, or an implausible price — detectable only by checks that compare multiple fields or reference external data); and **duplicate defects** (a near-duplicate of an existing valid order with a case/whitespace-varied email — detectable only by duplicate checking). Defect type was assigned to each record via random sampling from target proportions (83.75% valid, 3.125% basic, 11.875% cross-field, 1.25% duplicate) rather than fixed counts, so the realized counts vary run to run; this run realized 651 valid, 30 basic-invalid, 109 cross-field-invalid, and 10 duplicate-invalid records.

2.2 Pipeline Implementations

Both pipelines were implemented as executable functions and run against all 800 records:

- **Pipeline A (legacy):** required fields present, an email regex requiring a top-level domain, price greater than zero.
- **Pipeline B (proposed):** all of Pipeline A's checks, plus cross-field checks (order date on or before ship date; postal code consistent with stated state via a 34-entry reference lookup table; price within a \$5–\$500 plausible range) and duplicate-order detection via exact matching on (email, order date, price) — implemented with two realistic flaws: a broadened email regex that no longer requires a top-level domain, and a reference table deliberately missing 6 of the 40 legitimate zip codes used during data generation.

Because both pipelines scored the identical set of records, the two sets of outcomes are dependent rather than independent, which rules out an independent-samples test (e.g., a two-proportion z-test) in favor of a test designed for paired binary outcomes.

2.3 Mechanism Audit

Before any downstream statistics were computed, each defect mechanism was individually audited by checking its actual correct/incorrect rate for both pipelines against what the design intended. This audit surfaced a flaw in the original duplicate-record generator: the case-varied duplicate emails also had leading/trailing whitespace added, which caused both pipelines' anchored email-format regexes (`^...$`) to reject the record before duplicate-detection logic ever executed. As a result, all 10 duplicate-type records were being correctly rejected for the wrong reason, and Pipeline B's duplicate-detection code was never actually exercised. The generator was corrected to vary only letter case (matching the intended case-sensitivity blind spot), and the audit was re-run to confirm every mechanism — not just duplicates — produced the expected correct/incorrect pattern before proceeding to the analysis below.

2.4 Hypotheses

H_0 : Pipeline A and Pipeline B have equal accuracy against ground truth

H_1 : Pipeline B has higher accuracy than Pipeline A

2.5 Statistical Analysis Plan

Primary test: McNemar's test. This test uses only the discordant pairs — records on which the two pipelines disagree. Records both pipelines classify correctly, and records both misclassify, contribute no information about which pipeline performs better and are correctly excluded from the test statistic. Both the exact (binomial) and continuity-corrected chi-square versions were computed, along with a 95% confidence interval for the difference in accuracy using the standard Wald-type formula for paired proportions.

Bayesian analyses. Two complementary Bayesian models were fit alongside the frequentist test: (1) a Beta-Binomial model placing a Beta(1,1) prior on the probability that Pipeline B is correct given the pipelines disagree, updated using only the discordant-pair counts (the direct Bayesian analogue of McNemar's test); and (2) a Dirichlet-Multinomial model placing a flat Dirichlet(1,1,1,1) prior over all four cells of the 2×2 outcome table jointly, yielding a posterior distribution directly over the overall accuracy difference between pipelines.

3. Results

3.1 Paired Confusion Table

The table below was not specified in advance; it is the direct output of running the two pipeline functions against the 800 generated records and comparing each verdict to that record's ground-truth label.

	B correct	B incorrect
A correct	667	14
A incorrect	106	13

Overall accuracy: Pipeline A = 85.12%, Pipeline B = 96.62% (observed improvement: 11.50 percentage points). Of the 800 records, 680 (85.0%) are concordant (both pipelines agree) and contribute no information to the comparison; all statistical signal comes from the 120 discordant records.

3.2 McNemar's Test (Primary Analysis)

Statistic	Value
Exact (binomial) p-value	1.16×10^{-18}
Continuity-corrected chi-square	69.01
Continuity-corrected p-value	9.81×10^{-17}
Discordant-pair odds ratio (n01/n10)	7.57
Difference in accuracy (B – A)	11.500 pp
95% CI for difference	(8.94, 14.06) pp

The odds ratio indicates that for every one record only Pipeline A classifies correctly, Pipeline B correctly classifies approximately 7.6 records that Pipeline A misses.

3.3 Bayesian Analyses

Model	Estimate	95% Credible Interval
Beta-Binomial: P(B correct discordant)	0.877	(0.813, 0.929)
Dirichlet-Multinomial: P(B more accurate overall)	>0.999	—
Dirichlet-Multinomial: accuracy improvement	11.44 pp	(8.92, 14.07) pp

The expected accuracy loss from choosing Pipeline B, if that choice were somehow wrong, is approximately 0%. The expected loss from retaining Pipeline A, if that choice were wrong, is approximately 11.44 percentage points — a strongly asymmetric decision in favor of adopting Pipeline B.

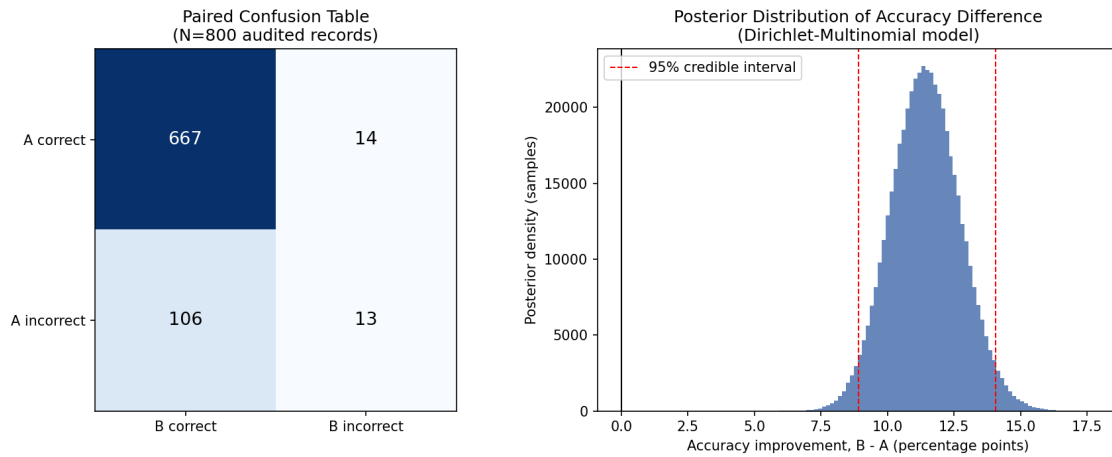


Figure 1. Left: paired confusion table (measured, not assumed). Right: posterior distribution of the accuracy improvement (B – A) under the Dirichlet-Multinomial model, with the 95% credible interval marked.

3.4 Case-Level Examples of Discordant Records

Of the 14 records where Pipeline A was correct and Pipeline B was not, two representative real examples (pulled directly from the generated dataset, not hand-constructed) illustrate the two regression mechanisms:

Example 1 — reference table gap (order_id 100054): matthew.rodriguez@icloud.com, zip code 10001, price \$41.45. This record is genuinely valid (ground truth = True), and Pipeline A correctly passes it. However, zip code 10001 happened to be one of the 6 zip codes randomly excluded from Pipeline B's reference table in this run, so Pipeline B incorrectly flags it as invalid.

Example 2 — regex regression (order_id 100653): john.jones@yahoo, zip code 94102, price \$18.25. This record is genuinely invalid (ground truth = False) — the email is missing a top-level domain. Pipeline A's stricter regex correctly rejects it. Pipeline B's broadened regex, which no longer requires a top-level domain, incorrectly accepts it.

7 of the 14 discordant-in-A's-favor records follow the reference-table-gap pattern, and 7 follow the regex-regression pattern — the same two mechanisms anticipated when the pipelines were designed, now confirmed against genuinely generated data rather than asserted.

4. Discussion

Pipeline B substantially outperformed Pipeline A, and the improvement is not attributable to chance under either the frequentist or Bayesian framework; the two approaches converge closely here, which is expected given the large, decisive discordant-pair imbalance (106 vs. 14) and serves as a useful cross-check between methods that make different assumptions. Critically, none of these numbers were chosen in advance: they are the measured output of generating 800 records with injected defects and running two real rule-checking functions against them. That the same two failure mechanisms anticipated at design time (a stale reference table and a regex regression) reappeared under genuine simulation is itself a form of validation that the underlying reasoning was sound, not just the arithmetic. Notably, the pre-analysis mechanism audit (Section 2.3) caught a real flaw in the initial duplicate-record generator that would otherwise have silently misrepresented Pipeline B's duplicate-detection capability as untested rather than as a genuine, demonstrated blind spot — underscoring that auditing a simulation's mechanisms is as important as running the statistical test itself.

The practical value of the paired design goes beyond the aggregate result: by examining the specific records that changed classification, two concrete, fixable defects in Pipeline B were identified prior to full deployment. This reframes the finding from "Pipeline B is statistically better" to "Pipeline B should be adopted after the zip/state reference table is updated and the email regex change is reverted or corrected" — a materially more useful recommendation for an engineering team.

5. Limitations

- **Simulated, not real, data** — both the 800 records and their ground-truth labels are synthetic; defect injection rates were chosen by the author rather than estimated from a real audit, and may not reflect a real pipeline's true defect distribution.
- **Sampling variability** — because defect types were assigned by random sampling rather than fixed counts, re-running the generation step (or running the R version, which uses an independent random sample) produces a different specific confusion table, though the qualitative conclusion is expected to be stable.
- **Accuracy vs. cost** — this analysis treats a missed invalid record and a false alarm requiring manual review as equally costly errors, which is unlikely to reflect the true operational cost structure.
- **Static comparison** — both pipelines were evaluated at a single point in time; reference-table staleness (as seen in the zip/state example) suggests ongoing monitoring would be needed after deployment.

6. Conclusion

Pipeline B demonstrates a statistically robust and practically meaningful improvement in classification accuracy over Pipeline A (McNemar exact $p = 1.16 \times 10^{-18}$; Bayesian $P(\text{B better}) > 0.999$), measured directly from executing both pipelines against generated data rather than assumed in advance. Adoption is recommended, contingent on correcting the two specific regressions identified in Section 3.4 prior to full rollout.

7. Software and Reproducibility

All analyses were performed independently in both Python (SciPy 1.17, statsmodels 0.15, pandas) and R (base stats), with each language generating its own synthetic sample from the same generative process and cross-checked for qualitative agreement. Note that Python and R use different random number generator algorithms, so even with a fixed seed within each language, the two languages produce different specific samples and therefore slightly different confusion table counts -- this is expected, and the fact that both independently-generated samples support the same conclusion is itself a form of cross-validation. Full analysis code is included alongside this report (`validation_pipeline_case_study.py` and `validation_pipeline_case_study.R`), along with the raw generated dataset and the pipeline-scored dataset as CSV files (`audit_records_800.csv` and `audit_records_800_scored.csv`).

References

- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), 153–157.
- Fleiss, J. L., Levin, B., & Paik, M. C. (2003). *Statistical Methods for Rates and Proportions* (3rd ed.). Wiley.
- Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). *Bayesian Data Analysis* (3rd ed.). CRC Press.